Analog to Digital Conversion: Part 2

ECE 2564, Embedded Systems

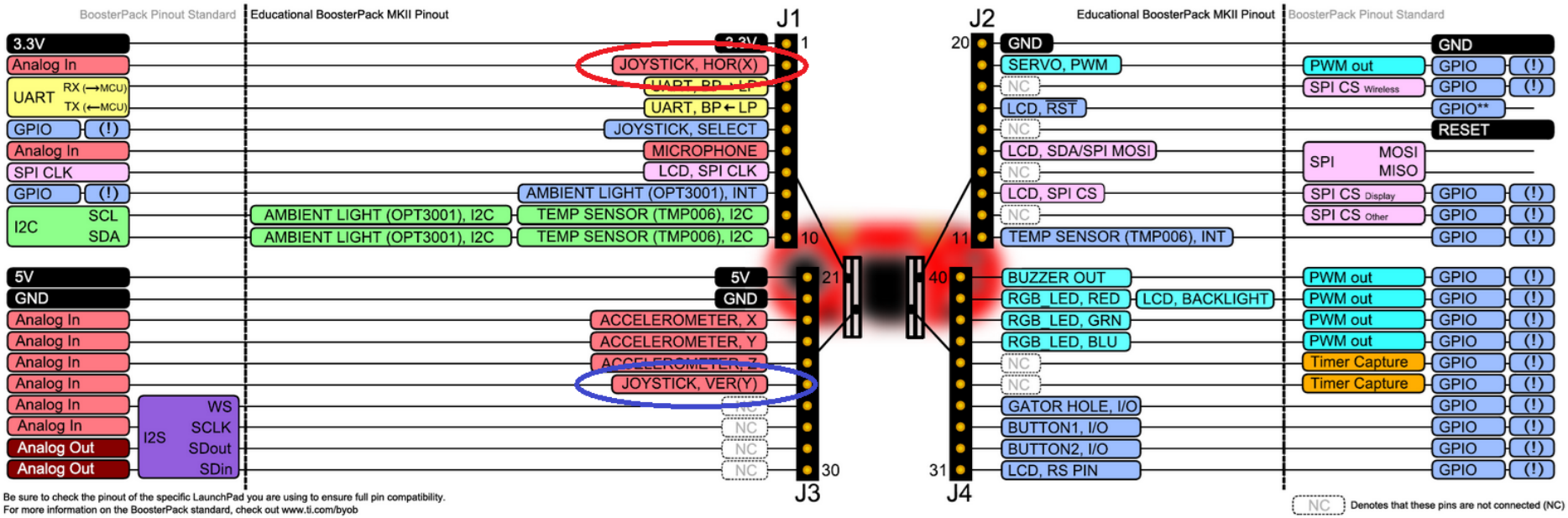
Fall 2025

ADC on the MSP432

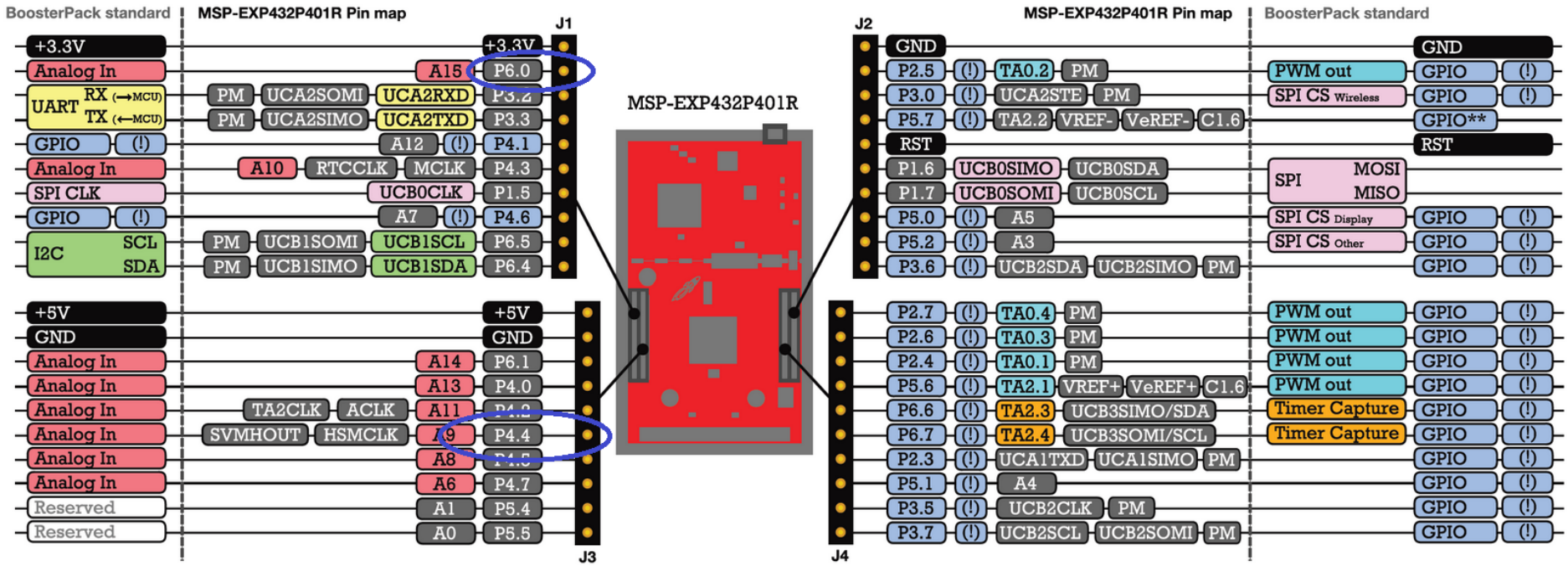
We'll be learning to use the MSP432 ADC capability with the joystick.

Import **this** repository

Where is the Joystick on the BoosterPack?



What Ports/Pins are Associated with the Joystick?



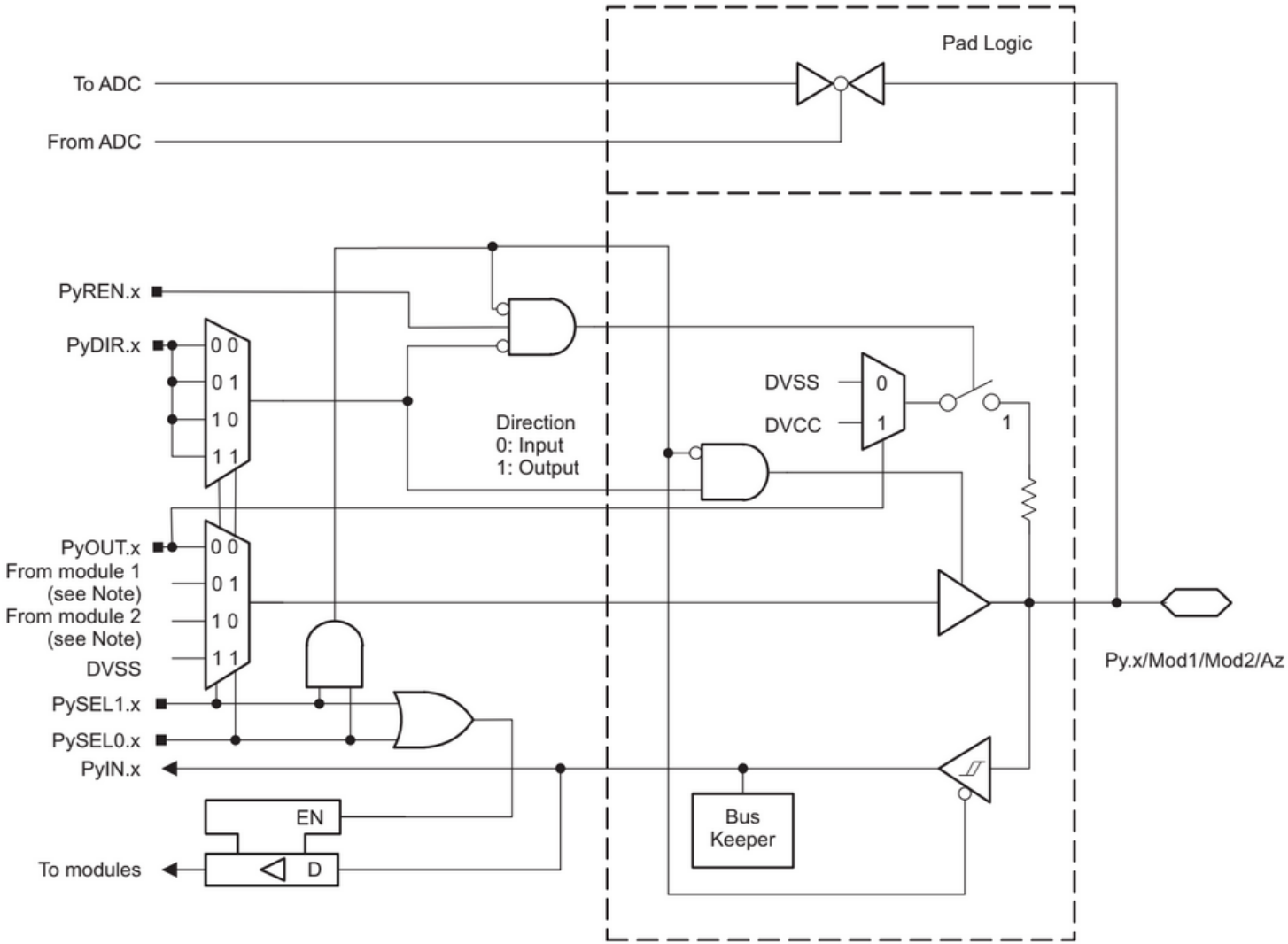
Joystick (X): P6.0; Joystick (Y): P4.4

Joystick

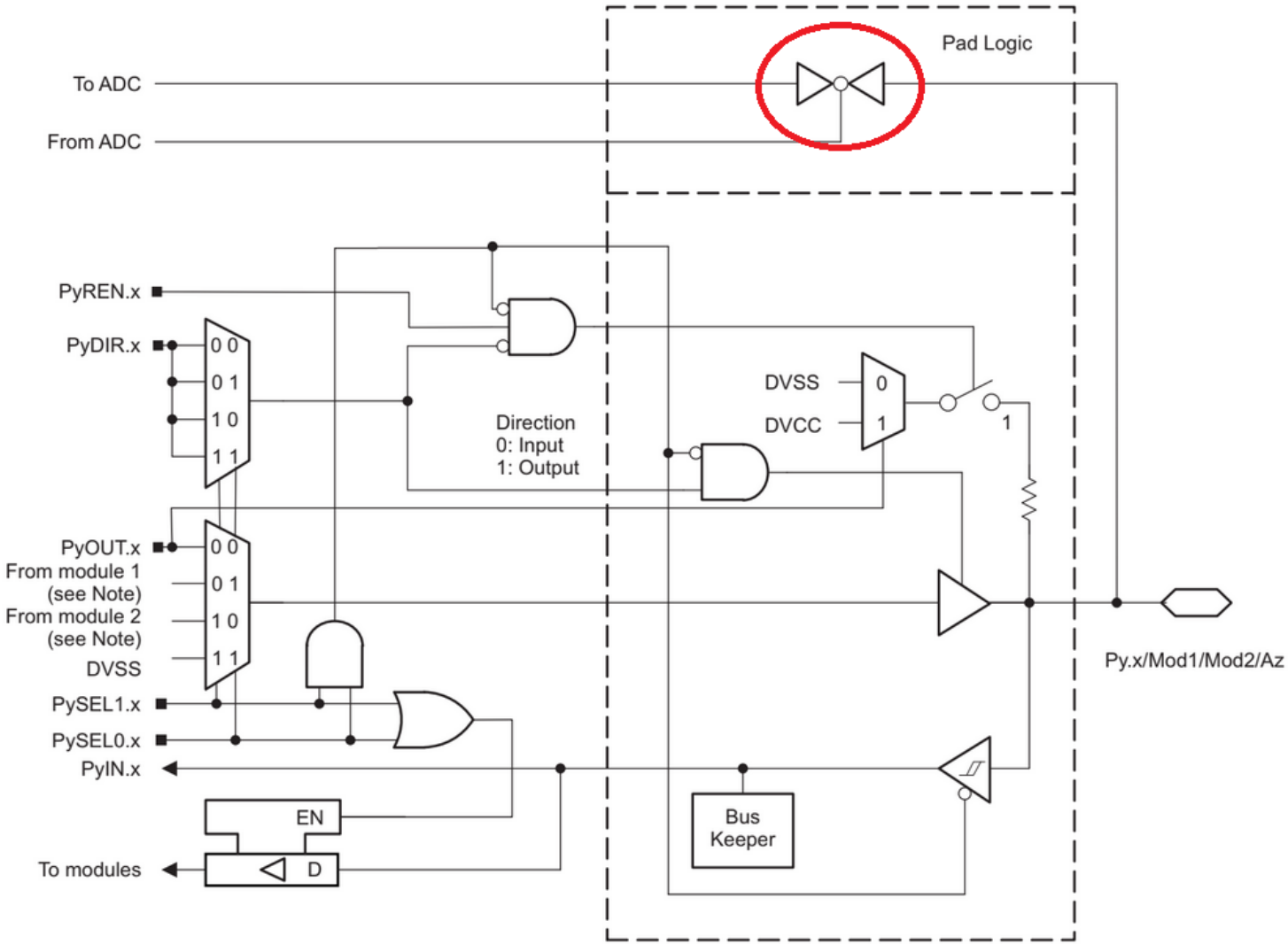
- Horizontal (X)
  - See the second row on the left on the BoosterPack diagram
  - It is P6.0 on the LaunchPad
  - It is also known as A15
- Vertical (Y)
  - P4.4 on the LaunchPad
  - Is is also known as A9

- Note:
  - The repository code only works for the X, horizontal, value
  - You'll need to program for Y, vertical, as well

Ports 4.4 and 6.0 Schematic



What is that Bowtie?



What is the Bowtie?

- The bowtie symbol represents a transmission gate
- You can learn about transmission gate here: **https://en.wikipedia.org/wiki/Transmission_gate**
- A transmission gate can conduct a signal in either direction depending on the control signal
- When the enable signal is active the analog signal can pass from the pin to the ADC

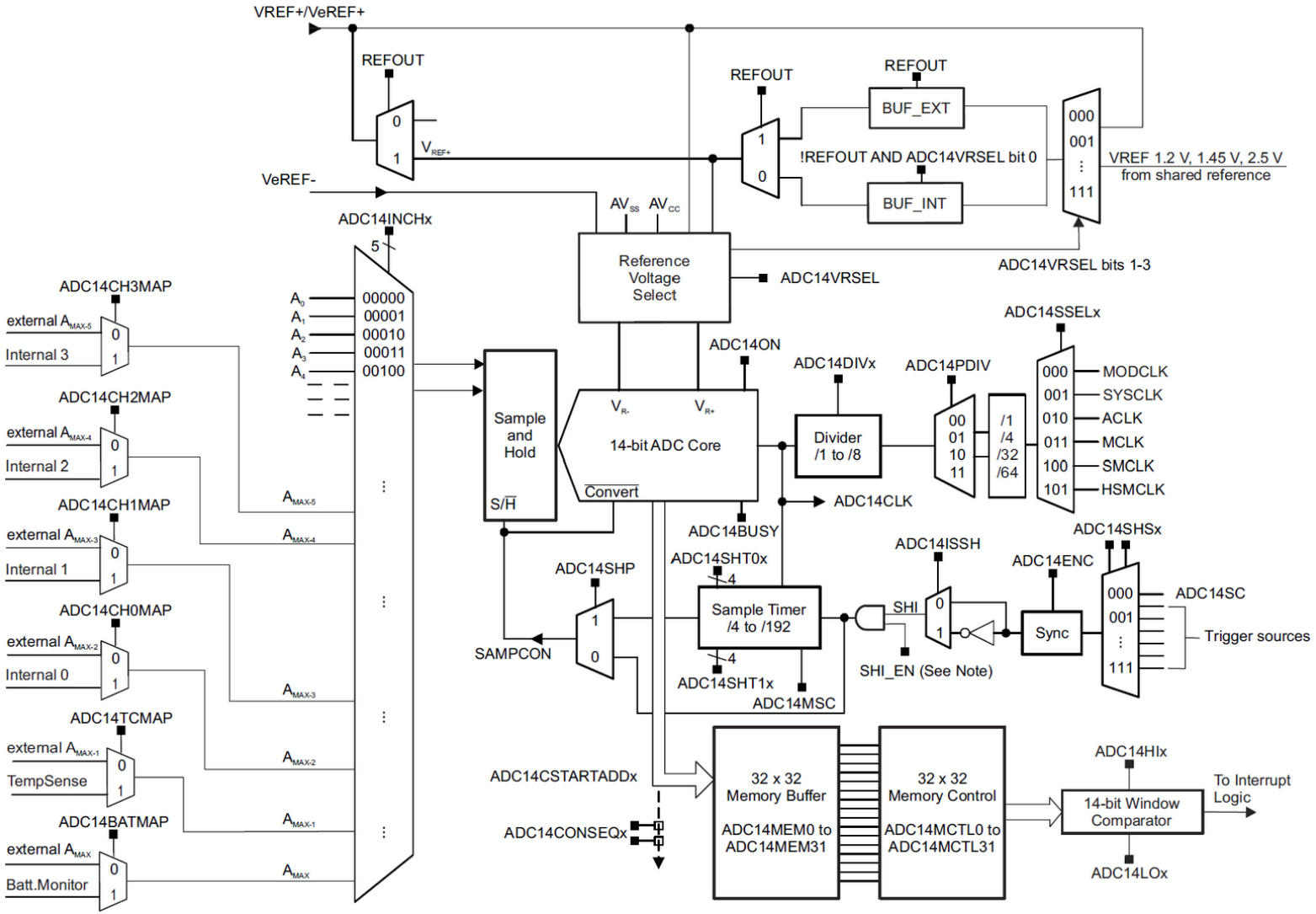
Setting for SEL0 and SEL1

P4SEL0.4 = 1, P4SEL1.4 = 1; P6SEL0.0 = 1, P6SEL1.0 = 1

PIN NAME (P6.x)	x	FUNCTION	CONTROL BITS OR SIGNALS[1]		
			P6DIR.x	P6SEL1.x	P6SEL0.x
P6.0/A15[2]	0	P6.0 (I/O)	I: 0; O: 1	0	0
		N/A	0	0	1
		DVSS	1		
		N/A	0	1	0
		DVSS	1		
		A15[3]	X	1	1
P4.4/HSMCLK/SVMHOUT/ A9	4	P4.4 (I/O)	I: 0; O: 1	0	0
		N/A	0	0	1
		HSMCLK	1		
		N/A	0	1	0
		SVMHOUT	1		
		A9[3]	X	1	1

Footnote (3): These settings for the selection pins disable the output driver and the input Schmitt trigger to prevent parasitic cross currents when applying analog signals.

“Precision” ADC Block Diagram



The Joystick Basic Example

Let's run the example to see what happens

Programming the ADC

1. Configure the relevant pins to operate as analog input channels
   - JOY (Y): analong input A9
   - JOY (X): analog input A15
2. Configure one or more analog input channels to be used
   - When a single channel is used the ADC will work in *single-channel* mode
   - When multiple channels are used the ADC will work in *multi-channel* mode
     - The ADC will alternate between the different pins while performing the A/D conversions
3. Configure the output memory buffer locations that will be used during the conversion
   - For example, Channel 9 will use ADC MEM0 and Channel 15 will use ADC MEM1
   - There are 32 memory buffer locations that can be used

Programming the ADC

- Decide on the conversion repetition strategy
- A full ADC conversion works through the following sequence:
  - The ADC is *enabled*, i.e., the A/D hardware is powered up
  - The ADC is *triggered*, i.e., the A/D hardware received the go-ahead to start a conversion
  - The ADC selects a channel and runs the sample-and-hold unit to read the input voltage from that channel
  - The ADC runs the SAR converter to determine the binary output code for the digital output
- The user has control over the sequence in many different ways

The Joystick Basic Example: main()

```
int main(void){
    Graphics_Context g_sContext;
    initialize();
    InitGraphics(&g_sContext);
    draw_Base(&g_sContext);
    unsigned vx, vy;
    while (1)
    {
        getSampleJoyStick(&vx, &vy);
        bool joyStickPushedtoRight = false;
        bool joyStickPushedtoLeft = false;
        drawXY(&g_sContext, vx, vy);
        if (vx < LEFT_THRESHOLD)
        {
            joyStickPushedtoLeft = true;
        }
        MoveCircle(&g_sContext, joyStickPushedtoLeft,
             joyStickPushedtoRight);
     }
}
```

The Joystick Basic Example: initialize()

```
void initialize()
{
    // stop the watchdog timer
    WDT_A_hold(WDT_A_BASE);

    initADC();
    initJoyStick();
    startADC();
}
```

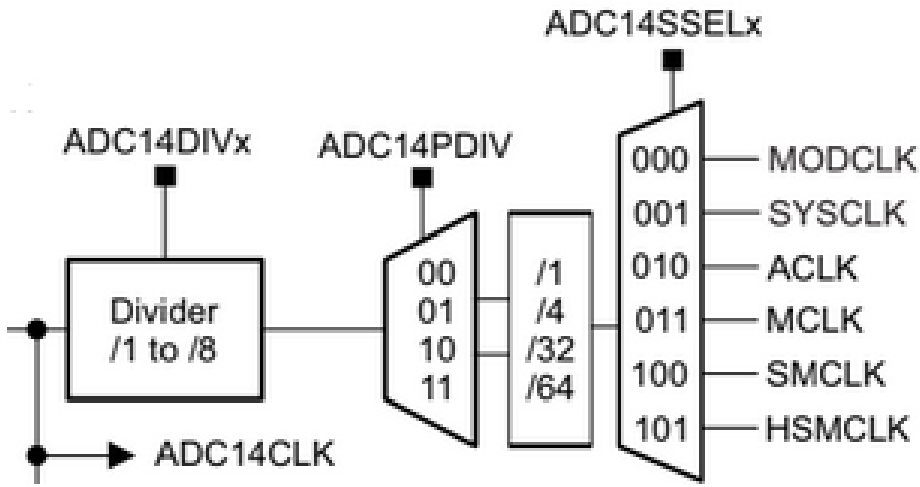
Programming the ADC

- Many things need to be done
  - Which signals should be initialized?
  - What conversion mode will be used and how are conversions made to occur?
  - How are the converted values read?
- Start with `initialize`

Programming the ADC — the Clock

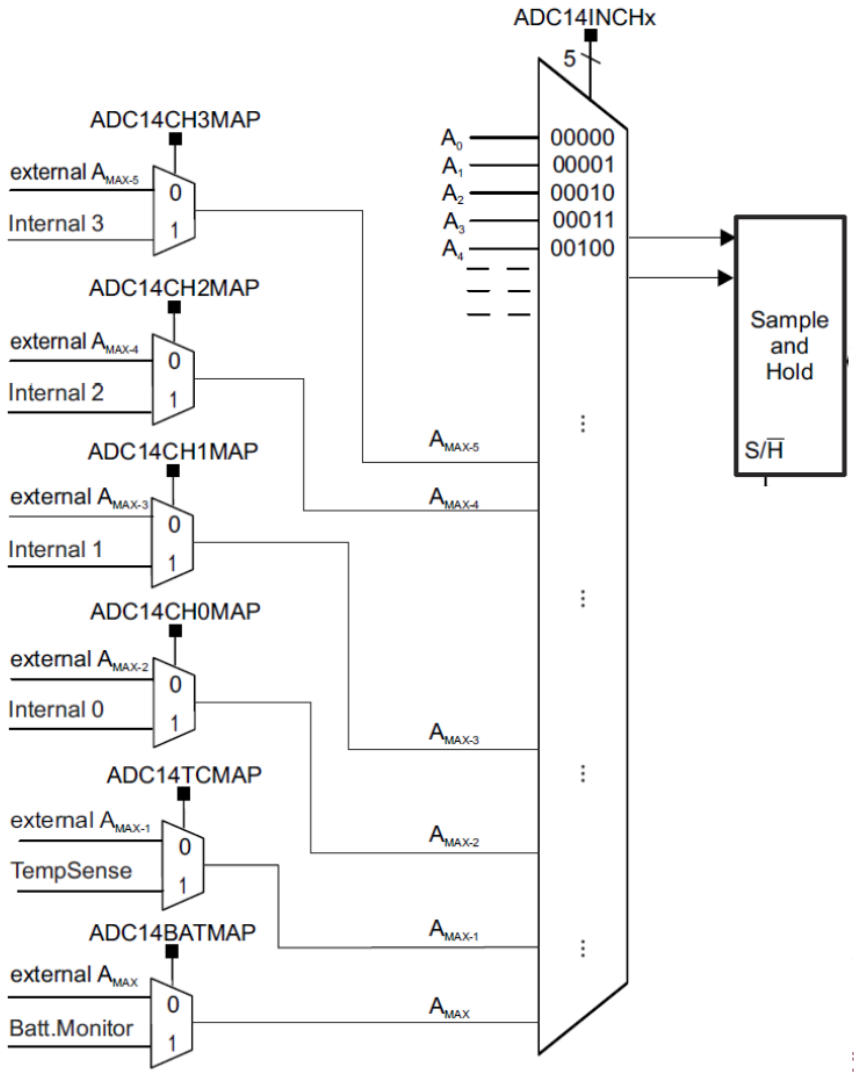
- The ADC needs a clock because it has to decide how often to *sample* an input
- Like some other peripherals the ADS has many choices for an input clock
  - In this course we use SYSCLK for the input clock for most peripherals

- There are two levels of clock dividers
  - Level 1: 1, 4, 32, 64
  - Level 2: 1, 8



Programming the ADC — the Multiplexers

- The huge mux and all the small multiplexers in front of it decide what analog signal is passed to the Sample/Hold unit to be converted
- Each small mux allows either an external analog signal to pass or an internal signal to pass



Initializing the ADC

```
void initADC() {
    ADC14_enableModule();

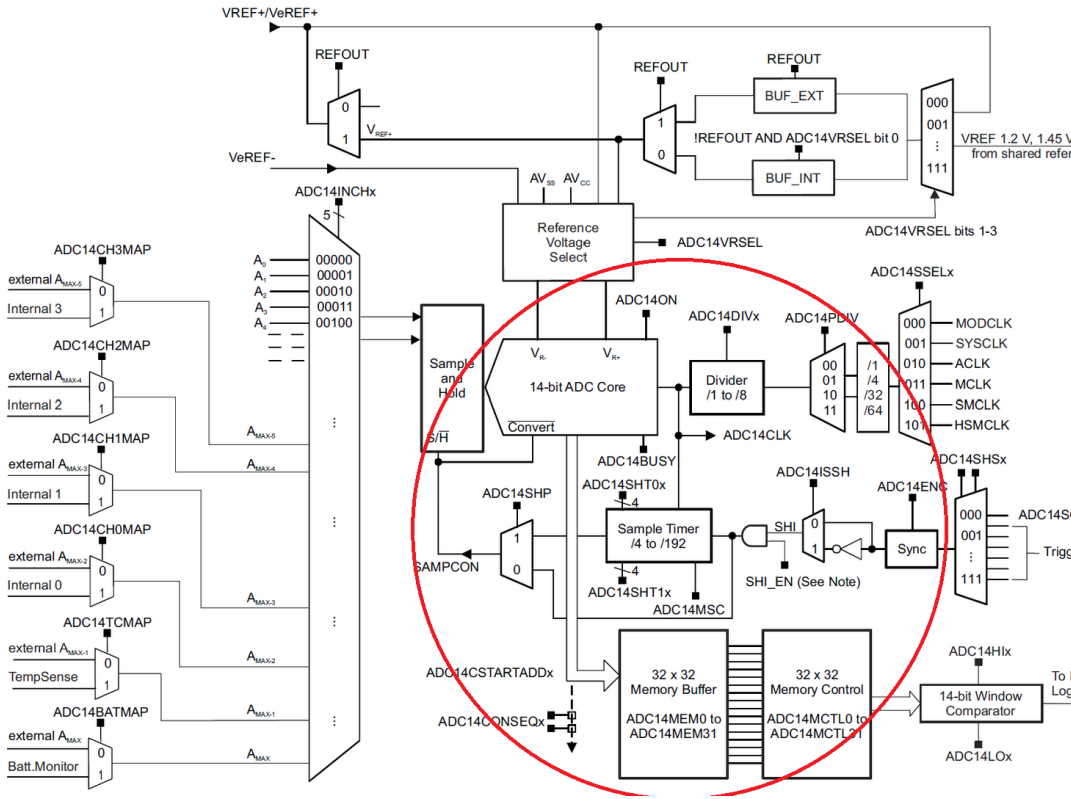
    ADC14_initModule(ADC_CLOCKSOURCE_SYSOSC,
                     ADC_PREDIVIDER_1,
                     ADC_DIVIDER_1,
                      0 );

    // This configures the ADC to store output results
    // in ADC_MEM0 for joystick X.
    // Todo: if we want to add joystick Y, then, we have
    // to use more memory locations
    ADC14_configureMultiSequenceMode(ADC_MEM0, ADC_MEM0, true);

    // This configures the ADC in manual conversion mode
    // Software will start each conversion.
    ADC14_enableSampleTimer(ADC_AUTOMATIC_ITERATION);
}
```

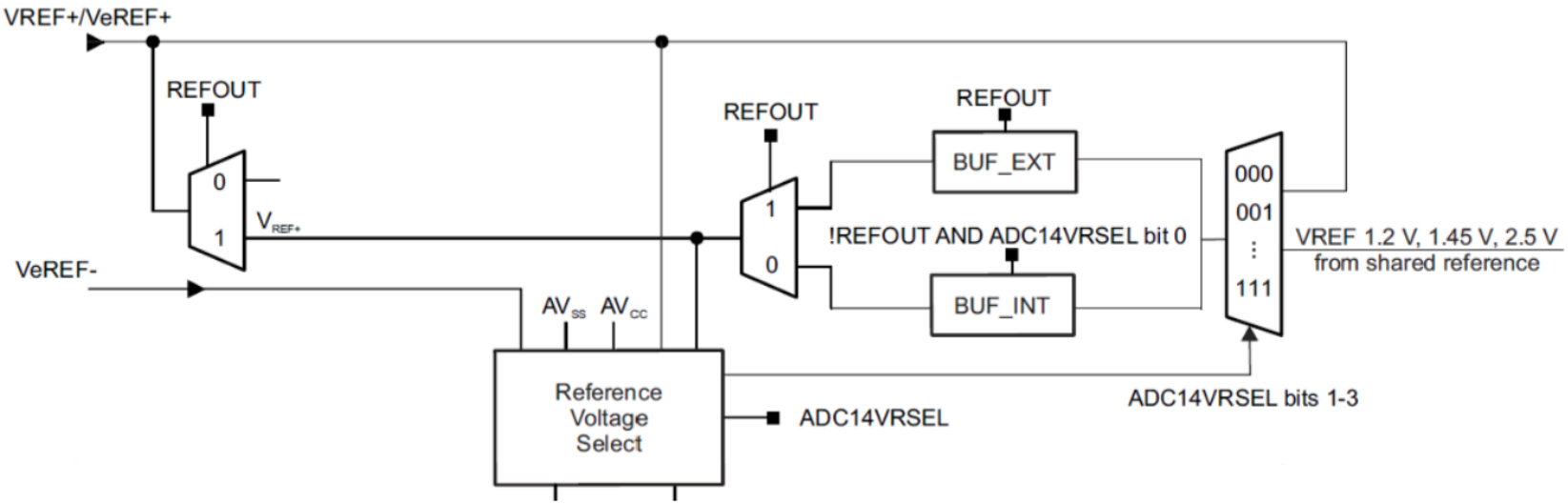
Initializing the ADC

- After the analog signal goes through Sample and Hold, the ADC core converts it to a 14-bit value
- The converted value is stored in one of 32 possible locations in a memory buffer
- The conversion can happen automatically one after another
- The conversion can be done manually by asserting a certain signal



Additional ADC Configurations for the Joystick

- We have to specify Vref+ and Vref-. We can use Vcc and Vss from the board or we can use external references.
- We have to specify the analog input we want to convert
- We have to specify if we are converting a single analog input or the *difference* of two neighboring analog inputs
- We have to specify the memory location or locations to store the results



Additional ADC Configurations for the Joystick

```
void initJoyStick() {

  // Store the result from input channel A15 (Joystick X)
  // into ADC_MEM0, in non-differential input mode, i.e
  // single input pin with using VSS and VCC
  ADC14_configureConversionMemory(ADC_MEM0,
                                  ADC_VREFPOS_AVCC_VREFNEG_VSS,
                                  ADC_INPUT_A15,  // joystick X
                                  ADC_NONDIFFERENTIAL_INPUTS);

  // This selects the GPIO as analog input
  // A15 is multiplexed on GPIO port P6 pin PIN0
  GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P6,
                                             GPIO_PIN0,
                                GPIO_TERTIARY_MODULE_FUNCTION);

  // TODO: add joystick Y
}
```

Let's Add Joystick (Y)

```
// TODO: add joystick Y
ADC14_configureConversionMemory(ADC_MEM1,
                                ADC_VREFPOS_AVCC_VREFNEG_VSS,
                                ADC_INPUT_A9,  // joystick Y
                                ADC_NONDIFFERENTIAL_INPUTS);


GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P4,
                                           GPIO_PIN4,
                                GPIO_TERTIARY_MODULE_FUNCTION);
```

Starting the ADC

```
void startADC() {
    // Starts the ADC with the first conversion
    // in repeat-mode, subsequent conversions run
    //automatically

    ADC14_enableConversion();
    ADC14_toggleConversionTrigger();
}
```

That completes the three functions in `initialize()`

Application Loop

```
while (1)
  {
     getSampleJoyStick(&vx, &vy);
...
}
```

```
void getSampleJoyStick(unsigned *X, unsigned *Y) {
  // ADC runs in continuous mode, we just read the conversion
  *X = ADC14_getResult(ADC_MEM0);

  // TODO: Read the Y channel
}
```

Summary

- Learned about ADC Hardware
- Wrote ADC software

- Understood how to look at the reference manual diagrams to determine which parts need to be configured/initialized